

Enhancing Human-Computer Interaction with Hand Mapping

A MINI PROJECT REPORT

Submitted by

Venkatesh M(920423243052)

Harish Gowtham B (920423243019)

Rajesh B (920423243301)

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE



KAMARAJ COLLEGE OF ENGINEERING AND TECHNOLOGY
(An Autonomous Institution - Affiliated to Anna University, Chennai)

K.VELLAKULAM, NEAR VIRUDHUNAGAR - 625 701

APRIL 2026

KAMARAJ COLLEGE OF ENGINEERING AND TECHNOLOGY
(An Autonomous Institution– Affiliated to Anna University, Chennai)
K.VELLAKULAM, NEAR VIRUDHUNAGAR – 625 701

BONAFIDE CERTIFICATE

Certified that the project report “Enhancing Human-Computer Interaction with Hand Mapping” is the bonafide work of “VENKATESH M – 920423243052, HARISH GOWTHAM B – 920423243019, RAJESH B – 920423243301” who carried out the project work under my supervision.

SIGNATURE

Dr.R.Ramya,
Head of the Department,
Associate Professor,
Dept. of Artificial Intelligence & Data
Science,
Kamaraj College of Engg &
Tech,K.Vellakulam, Near Virudhunagar
625 701.

SIGNATURE

Mrs.K.MuthuLakshmi ,
Assistant Professor Professor ,
Dept. of Artificial Intelligence & Data
Science,
Kamaraj College of Engg & Tech,
K.Vellakulam, Near Virudhunagar
625 701.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ABSTRACT

This project presents a web-based hand gesture control system designed to improve the way users interact with computers during presentations, demonstrations, and training sessions. Instead of relying only on keyboard shortcuts, remote controls, or mouse clicks, the proposed system uses real-time webcam input to detect hand landmarks and convert gestures into meaningful computer actions. The goal is to make interaction faster, more natural, and less interruptive, especially in situations where the speaker must stay focused on the audience. The system combines computer vision and modern web technologies in a single application. MediaPipe Hands identifies hand landmarks efficiently, OpenCV captures and prepares frames, FastAPI manages backend communication, WebSocket provides real-time updates, and PyAutoGUI simulates keyboard actions on the operating system. A browser-based dashboard displays the live camera feed, detected gesture, action history, and control statistics so that the user can understand what the system is doing at every step. The project is important because it demonstrates a practical approach to touchless interaction using affordable hardware and open-source tools. It supports usability, accessibility, and workflow continuity while keeping the interface simple enough for non-technical users. The report also discusses the problem statement, literature review, system design, implementation approach, results, limitations, and future scope of the proposed system.

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our Principal, Dr. S. Senthil, M.E., Ph.D., for granting us permission and providing the opportunity to carry out this project within our college premises. His encouragement and support have been a great source of motivation throughout the successful completion of this work.

We would also like to convey our heartfelt thanks to our Head of the Department, Dr. R. Ramya, M.E., Ph.D., for her constant encouragement, valuable suggestions, and continuous support throughout the project work. Her guidance and motivation greatly helped us in completing the project successfully.

We sincerely express our deep sense of gratitude to our respected project guide, Mrs.K.MuthuLakshmi,M.E.,Assistant Professor, Kamaraj College of Engineering and Technology, Virudhunagar, for her valuable guidance, constructive suggestions, and expert opinions during every stage of the project. Her encouragement, patience, and continuous supervision played an important role in shaping this project in the best possible manner.

We also extend our thanks to all the teaching and non-teaching staff members of the department for their support and cooperation whenever required. Their assistance and encouragement were highly valuable during the course of this project.

TABLE OF CONTENTS

Section	Title	Page No
	ABSTRACT	iii
	ACKNOWLEDGEMENT	iv
	LIST OF TABLES	vii
	LIST OF FIGURES	Vii
	LIST OF FLOW CHART	Vii
	LIST OF SYMBOLS AND ABBREVIATIONS	viii
1	INTRODUCTION	1
1.1	Overview	1
1.2	Problem Statement	2
1.3	Importance of the Work	3
1.4	Objectives	4
1.5	Social Impacts	5
1.6	Challenges	5
1.7	Scope and Motivation	6
2	LITERATURE REVIEW	7
2.1	Gesture Recognition in HCI	7
2.2	Related Tools and Techniques	8
2.3	Existing Gesture-Based Systems	9
2.4	Research Gap	11
2.5	Summary of Literature Review	13
3	METHODOLOGY	14
3.1	System Requirements	14
3.1.1	Hardware Requirements	15

3.1.2	Software Requirements	15
3.2	System Architecture	16
3.3	Flow Diagram	17
3.4	Gesture Detection & Classification	18
3.5	List of Modules	19
3.6	Modules Description	20
3.7	Gesture Mapping Rules	21
3.8	Data Flow and State Handling	22
4	RESULTS AND DISCUSSION	23
4.1	Screenshots	23
4.2	Results Obtained	24
4.3	Discussion	25
4.4	Advantages	26
4.5	Limitations	26
4.6	Performance Observation Table	27
5	CONCLUSION AND FUTURE SCOPE	28
5.1	Conclusion	28
5.2	Future Scope	29
5.3	Final Remark	29
6	REFERENCES	30
7	MAPPING	33
7.1	PO Mapping	34
7.2	PSO Mapping	36
7.3	SDG Mapping	37
8	CONFERENCE DETAILS	38

LIST OF TABLES

Table No	Title	Page No
2.1	Key findings from literature	10
2.4.1	Hand Mapping Tools and Source	12
3.1	Module Summary	20
4.1	Performance Observation Table	29

LIST OF FIGURES

Fig No	Title	Page No
3.1	System Architecture	15
3.2	Flow Diagram	17
4.1	Dashboard Interface	23

LIST OF FLOW CHARTS

Flow Chart No	Title	Page No
3.1	Workflow of System	17

LIST OF SYMBOLS AND ABBREVIATIONS

Abbreviation	Meaning
HCI	Human-Computer Interaction
AI	Artificial Intelligence
UI	User Interface
API	Application Programming Interface
FPS	Frames Per Second
OS	Operating System
ML	Machine Learning
CV	Computer Vision
WebSocket	Real-time communication protocol

CHAPTER 1

INTRODUCTION

1.1 Overview

Human-computer interaction has changed steadily over the years, moving from command-line text entry to graphical interfaces, touch screens, voice assistants, and vision-based interaction. Even with these advances, many everyday tasks still depend on a keyboard and mouse. In classroom presentations, project demonstrations, and live seminars, that dependency often becomes a practical limitation because the user must constantly move between speaking, navigating slides, and controlling the computer. As a result, the interaction feels less natural and more distracting than it should be.

Hand gestures provide a promising alternative because they are a familiar and expressive form of communication. People naturally use hand movements to point, emphasize, indicate direction, and control attention. When those motions are interpreted by a computer system, the user gains a more direct and intuitive way to interact with software. This idea is particularly useful in presentation environments where the speaker should remain focused on the audience rather than on physical input devices.

The project titled 'Enhancing Human-Computer Interaction with Hand Mapping (Google AI Edge)' explores this idea through a web-based hand gesture control system. The application uses webcam input to detect hand landmarks in real time, classifies simple gestures such as next, previous, and home, and translates them into keyboard actions that control slide navigation or other desktop functions. A dashboard in the browser provides visibility into the live feed, gesture status, action history, and system statistics, making the solution practical and easy to observe during use.

The proposed system is intentionally designed to be lightweight and accessible. Instead of relying on specialized controllers or expensive hardware, it uses widely available components such as a standard camera and a web browser. This makes it suitable for student projects, classroom demonstrations, startup showcases, and small training rooms where cost and convenience matter. The system also demonstrates how edge-oriented AI tools can be combined with simple automation utilities to create a meaningful real-world application.

1.2 Problem Statement

Many presenters and computer users face difficulty when they need to control software while keeping their attention on the audience or on physical materials. Traditional methods such as keyboard shortcuts, presentation remotes, touchpads, and mouse clicks work well in ordinary use, but they often interrupt the flow of a live presentation. The speaker must either walk back to the computer, carry an extra remote, or memorize shortcuts that are not always convenient in the moment.

Another problem is that existing gesture-based systems are often either too simple or too isolated. Some solutions are only desktop scripts, which means they do not provide a polished user interface or detailed feedback. Other systems depend on dedicated devices, require difficult calibration, or do not show the user what gesture was recognized and why a particular action was triggered. This can create confusion and reduce trust in the system.

In many cases, gesture control also lacks a clear connection to real-time monitoring. When the user cannot see the camera feed, the landmark detection result, and the action log in one place, it becomes harder to confirm that the system is working correctly. There is therefore a need for a responsive browser-based system

that combines webcam capture, gesture recognition, action execution, and live dashboard feedback in a single solution.

- Dependence on keyboards and remotes during live sessions.
- Breaks in speaking flow when the user must switch attention to the computer.
- Lack of real-time feedback in many gesture prototypes.
- Extra cost and inconvenience caused by specialized hardware.
- Need for a simple and browser-friendly interface that is easy to observe.

1.3 Importance of the Work

This project is important because it demonstrates a practical form of touchless interaction that can be understood and used by a wide range of people. By converting a hand posture into a computer command, the system removes the need to repeatedly reach for the keyboard or mouse. That small change improves concentration, keeps the user in the flow of presentation, and creates a more polished experience for the audience.

The work is also valuable from a technical perspective. It combines real-time video processing, web communication, operating system automation, and interface design into one coherent system. Many student projects show gesture recognition as a model output, but fewer projects connect that recognition to a polished application dashboard that confirms the action, stores the history, and communicates the current state clearly. This project bridges that gap.

The system is also important because it shows how open-source tools can be used effectively for meaningful interaction design. MediaPipe helps with landmark detection, OpenCV handles camera frames, FastAPI supports asynchronous backend processing, WebSocket keeps the browser updated, and PyAutoGUI converts

recognized gestures into familiar keyboard events. This combination is cost-effective, modular, and easy to extend for future use cases.

1.4 Objectives

The main objective of the project is to design and develop a user-friendly web-based hand gesture control system that can detect gestures accurately, map them to useful commands, and display the output clearly in the browser. The project focuses on making gesture interaction practical for presentations and related tasks.

- To develop a user-friendly web-based platform for hand gesture control.
- To detect hand landmarks accurately using real-time webcam input.
- To classify gestures and map them to keyboard actions such as next, previous, and home.
- To provide a dashboard with live video, gesture output, and usage statistics.
- To implement a clean backend architecture with FastAPI and WebSocket communication.
- To support efficient performance using frame skipping, resizing, and cached detection results.
- To make the system simple enough for students, teachers, and non-technical users.

1.5 Social Impacts

The proposed system has a positive social impact because it encourages hands-free interaction and reduces dependence on physical input devices in routine tasks. For users with temporary or permanent movement restrictions, gesture-based control can make digital interaction more inclusive and reduce friction in everyday

use. In classrooms and conferences, the system also helps presenters maintain eye contact and communicate more naturally with the audience.

The project additionally supports affordable innovation. Since it can run with a normal webcam and a browser, it does not require specialized controllers or expensive accessories. This lowers the entry barrier for schools, small offices, startups, and learning labs. It also promotes a wider understanding of how AI-assisted edge tools can improve daily computing without making the interface overly complicated.

- Promotes accessible and hands-free interaction.
- Improves presentation quality and audience engagement.
- Reduces cost by using existing camera hardware.
- Supports digital inclusion for varied user needs.
- Encourages practical use of AI in small-scale environments.

1.6 Challenges

Although the project is useful, several challenges had to be considered during design and implementation. The first challenge is stable hand detection under different lighting conditions, camera qualities, backgrounds, and hand orientations. A system that works well only in controlled lighting would not be dependable in a real classroom or seminar room. The detection pipeline therefore needs to be robust enough to handle ordinary variations without producing too many false results.

A second challenge is performance. Real-time video processing can easily become heavy if every frame is processed at full resolution. To keep the interface responsive, the system must balance accuracy and speed using techniques such as frame skipping, resizing, and result reuse. Other challenges include preventing false

triggers, handling camera switching, keeping the dashboard responsive, and ensuring that the gesture mapping rules remain predictable and easy to understand.

- Ensuring data stability under varied lighting conditions.
- Reducing false gesture triggers caused by accidental movements.
- Balancing recognition accuracy with frame-rate performance.
- Maintaining responsiveness in the browser dashboard.
- Keeping the gesture-to-action logic simple and predictable.
- Supporting camera switching and temporary connection recovery.

1.7 Scope and Motivation

The scope of the project extends beyond slide navigation. The same framework can later support volume control, media playback, annotation tools, browser navigation, and accessibility-focused commands. Once the gesture engine and dashboard pattern are established, additional gestures can be mapped to many keyboard-driven actions with minimal structural change. This makes the system a strong base for future enhancement.

The motivation for the work comes from the idea that interaction should feel natural and should not distract from the task itself. In a seminar, a trainer should be able to move between slides without turning away from the audience. In an educational demo, a student should be able to control content while keeping attention on the class. This project transforms that need into a working application and demonstrates the practical use of computer vision for everyday interaction.

CHAPTER 2

LITERATURE REVIEW

Research in human-computer interaction has increasingly focused on making interaction more natural, fast, and contactless. In this direction, hand gesture recognition has become an important research area because it allows users to communicate with systems through movement rather than physical touch or traditional input devices. Recent survey work shows that gesture recognition is now widely used in applications such as AR/VR, accessibility, robotics, and smart interfaces, while major challenges still include real-time performance, robustness, and generalization across users and environments (Hand Gesture Survey, 2025).

Gesture recognition systems aim to convert hand motion into meaningful machine actions. Earlier systems often relied on gloves, sensors, or infrared hardware, but modern approaches use ordinary camera-based vision methods, which are easier to deploy and more cost-effective. The development of on-device hand tracking and real-time gesture recognition has made such systems more practical for everyday human-computer interaction (MediaPipe Hands, 2020; On-device HGR, 2021).

2.1 Gesture Recognition in Human-Computer Interaction

Gesture recognition translates physical hand movement into commands that a computer can understand. This makes interaction more natural because gestures are already part of human communication. In the literature, gesture-based input has been applied in multimedia control, gaming, assistive systems, and remote operation tasks. However, studies consistently show that the usefulness of such systems depends not only on detection accuracy but also on how well the system maps gestures to actions and communicates feedback to the user (Hand Gesture Survey, 2025; Gestop, 2021).

A major contribution in this area is **MediaPipe Hands (2020)**, which introduced a real-time on-device hand tracking pipeline using a palm detector and a hand landmark model. This work is important because it showed that accurate hand tracking can be achieved using a single RGB camera without requiring expensive hardware. The method has become a foundation for many later gesture-based HCI systems (MediaPipe Hands, 2020).

Another important direction is customization. Many systems originally supported only a fixed list of gestures, but later work showed that users often need gesture sets that match their own tasks. The paper **Vision-Based Hand Gesture Customization from a Single Demonstration (2024)** demonstrates that users can define bespoke gestures from one example, supporting one-handed, two-handed, static, and dynamic gestures. This is highly relevant for hand mapping systems because it shows that gesture-to-command mapping can be made more flexible and user-centered (Shahi et al., 2024).

2.2 Related Tools and Techniques

Several tools and techniques support practical gesture recognition systems. **OpenCV** is commonly used for video capture and frame processing, while **MediaPipe Hands** is used for landmark detection. Together, they form an efficient pipeline for extracting hand positions and finger movements in real time. The literature shows that this combination is widely used because it balances speed and accuracy for interactive applications (MediaPipe Hands, 2020; On-device HGR, 2021).

The paper **On-device Real-time Custom Hand Gesture Recognition (2023)** is especially useful because it focuses on low-code and no-code custom gesture building. It allows users to collect a small number of examples and train a customized gesture model that runs on-device. This is important for hand mapping

projects because it shows that gesture systems can be personalized instead of limited to predefined command sets (Uboweja et al., 2023).

Another practical contribution is **Gestop: Customizable Gesture Control of Computer Systems (2021)**. This work presented a gesture-control framework that lets users define and use gestures to control computers in real time. It is relevant because it connects gesture recognition directly to operating-system actions, which is exactly the kind of mapping needed in interaction systems like presentation control or desktop navigation (Gestop, 2021).

The paper **Deep learning based Hand gesture recognition system and design of a Human-Machine Interface (2022)** also contributes to this area by combining gesture recognition with a full control interface. The system uses hand detection, segmentation, deep learning classification, and a virtual mouse-style interface to control applications. This shows how gesture recognition can be extended beyond classification into practical human-machine interaction (Sen et al., 2022).

The **HaGRID dataset (2022)** is another key resource because gesture systems depend heavily on the quality and diversity of training data. HaGRID includes a very large number of images, multiple gesture categories, and many subjects, which helps improve recognition robustness. The dataset was created specifically to support hand gesture recognition and device-control tasks, making it a valuable benchmark for future work (HaGRID, 2022).

2.3 Review of Existing Gesture-Based Systems

Existing gesture-based systems can be grouped into three broad categories. The first category focuses mainly on recognition accuracy and model performance. The second category focuses on gesture-controlled applications such as virtual mouse systems or multimedia control. The third category combines recognition with

user customization, feedback, and system integration. Recent research shows that systems become more practical when they support both detection and interaction design rather than recognition alone (Hand Gesture Survey, 2025; Deep learning HGR HMI, 2022; Gestop, 2021).

The paper **Real-Time Hand Gesture Recognition: Integrating Skeleton-Based Data Fusion and Multi-Stream CNN (2024)** shows that skeleton-based and fused representations can improve dynamic gesture recognition while keeping computation efficient. This is important because real-world hand mapping systems often need to recognize movement over time, not just single static hand poses (Yusuf et al., 2024).

The study **A Systematic Study of Various Fingertip Detection Techniques for Air Writing (2023)** is relevant because it highlights the importance of fingertip tracking for gesture-based input. Air-writing and hand-gesture systems depend on accurate fingertip localization, especially when gestures are used for control or input drawing. This supports the idea that hand mapping systems need precise feature detection, not only whole-hand detection (Heena & Ranjan, 2023).

A more recent and advanced direction is **GestureGPT: Toward Zero-Shot Free-Form Hand Gesture Understanding with Large Language Model Agents (2023/2024)**. This work moves beyond fixed gesture sets and explores understanding of free-form gestures through context and language reasoning. It shows that the field is evolving from rigid command mapping toward more natural and intelligent gesture interpretation, which is an important future direction for HCI research (GestureGPT, 2024).

2.4 Research Gap

The literature shows that many gesture systems are technically successful but still incomplete from a user-experience point of view. Some systems focus only on recognition accuracy, while others focus only on command execution. In many cases, the system does not clearly explain what gesture was detected, how the gesture was mapped, or how fast the response was generated. This limits trust and usability in real interaction settings (Hand Gesture Survey, 2025; Gestop, 2021; Deep learning HGR HMI, 2022).

Another gap is adaptability. Several systems are built for one fixed gesture set or one specific use case, which makes them difficult to reuse. Later research on customization shows that users benefit when they can define or adjust gestures based on their own needs. This strongly supports the need for a hand mapping system that is configurable and easy to extend (Shahi et al., 2024; Uboweja et al., 2023; GestureGPT, 2024).

The present project addresses these gaps by focusing on **hand mapping** as a complete interaction process. It combines hand tracking, gesture interpretation, and command mapping in a way that is easier for users to understand and control. This makes the system more suitable for real HCI use, especially where clarity, speed, and flexibility are required (MediaPipe Hands, 2020; Gestop, 2021; HaGRID, 2022).

2.4.1 Hand Mapping Tools and Source

Source / Tool	Main Idea	Key Strength	Limitation
MediaPipe Hands	Real-time hand landmark detection using on-device vision models	Fast, lightweight, and accurate for common gestures	May need tuning for lighting and occlusion
OpenCV-based controllers	Capture webcam frames and process gesture input locally	Easy to integrate with Python applications	Desktop scripts often lack polished visualization
PyAutoGUI remotes	Convert a gesture or hotkey into operating system actions	Simple way to trigger slide navigation	Depends on correct timing and cooldown control
Web dashboard prototypes	Show live feed, logs, and controls in the browser	Better feedback and user understanding	Can become slow if streaming is not optimized
Edge AI interaction systems	Keep inference near the device for low latency	Reduces network dependence and supports responsiveness	Requires careful performance balancing

2.5 Summary of the Literature Review

The reviewed literature shows that hand gesture recognition is a strong and growing area in human-computer interaction. Studies on real-time hand tracking, customizable gestures, dataset design, and gesture-controlled interfaces demonstrate that vision-based interaction is now practical and useful in many settings (MediaPipe Hands, 2020; HaGRID, 2022; On-device HGR, 2023; Vision-Based Customization, 2024).

At the same time, the literature also shows that accuracy alone is not enough. Systems must also provide feedback, adaptability, and smooth integration with user applications. For that reason, an effective hand mapping system should be fast, stable, understandable, and easy to customize (Hand Gesture Survey, 2025; Gestop, 2021; GestureGPT, 2024).

These findings directly support the design of the present project, **Enhancing Human-Computer Interaction with Hand Mapping**, which aims to improve interaction through real-time gesture detection, clear command mapping, and user-friendly control.

CHAPTER 3

METHODOLOGY

3.1 System Requirements

The system requirements are intentionally modest so that the application can run on standard student or office machines. The project is designed to be practical rather than demanding, and therefore uses widely available software tools and common camera hardware.

3.1.1 Hardware Requirements

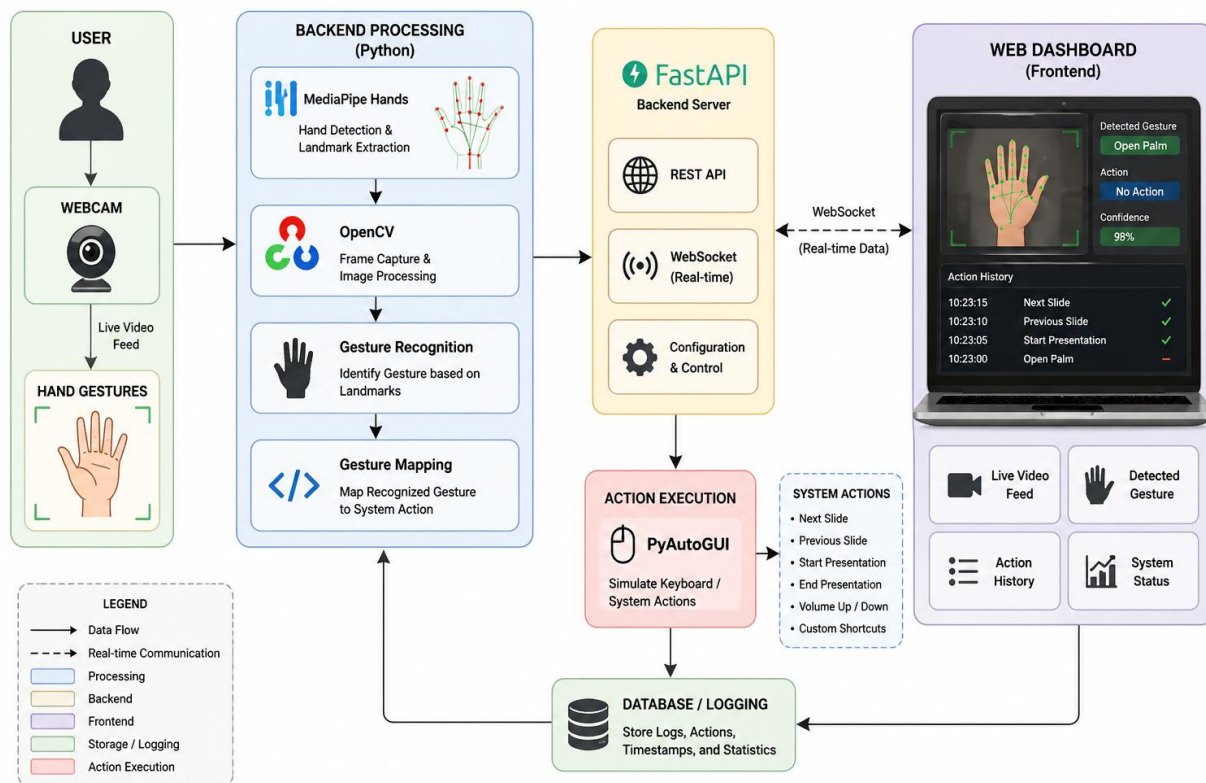
- Processor: Intel i3 or above, or any comparable modern CPU.
- RAM: Minimum 4 GB, with 8 GB recommended for smoother video handling.
- Storage: At least 500 GB for the operating system, project files, and dependencies.
- Input Device: Built-in laptop webcam or an external USB camera.

3.1.2 Software Requirements

- Backend: FastAPI with Uvicorn for asynchronous request handling and WebSocket streaming.
- Frontend: HTML, CSS, and JavaScript for the single-page dashboard.
- Computer Vision: OpenCV for camera capture and image processing.
- Gesture Engine: MediaPipe Hands for landmark detection and hand tracking.
- Keyboard Simulation: PyAutoGUI for sending presentation shortcut keys.
- Operating System: Windows, Linux, or macOS with Python support.

These requirements make the system easy to install and reproduce. They also support a broad range of demonstration environments, from classroom laptops to development workstations. Since the application is not tied to a specialized sensor or a cloud service, it can be configured quickly for local use.

3.2 System Architecture

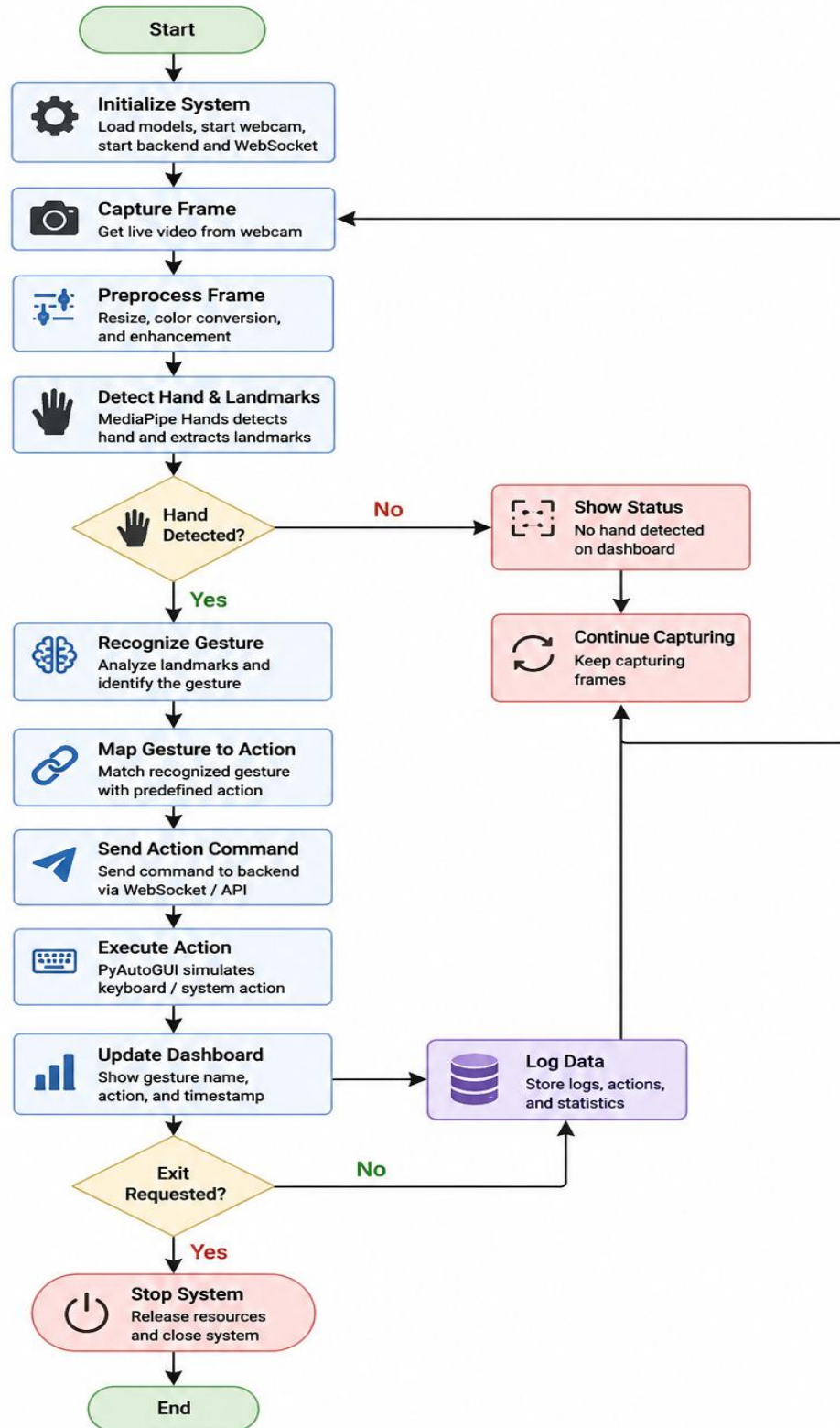


The system follows a layered architecture that separates interface, processing, and device interaction responsibilities. The presentation layer is responsible for the browser dashboard and the live user experience. The application layer manages the FastAPI routes, WebSocket communication, gesture logic, and slide control behavior. The data and device layer handles the webcam feed, landmark detection, keyboard automation, and configuration values.

This separation improves maintainability because each part can evolve without rewriting the whole project. If the gesture logic needs refinement, changes can be made in the backend layer. If the dashboard design needs adjustment, the frontend can be updated independently. If the system must support a different camera source, only the device configuration needs to be changed. Such separation is very useful in student projects because it keeps the code organized and easier to test.

The architecture also supports responsiveness. Rather than waiting for repeated page refreshes, the browser receives updates as soon as the backend produces them. The WebSocket channel carries status messages and action records, while the camera pipeline feeds the detection engine continuously. This creates a smoother interaction and allows the user to observe the system in real time.

3.3 Flow Diagram



The workflow of the project begins when the dashboard is opened in the browser. The backend initializes the camera source and starts streaming frames to the gesture engine. Each frame is resized and processed so that the recognition pipeline remains efficient. When the system detects a hand, it evaluates the landmark configuration and decides whether the gesture matches the defined mapping rules.

If the gesture is recognized, the application converts it into a keyboard shortcut by using PyAutoGUI. A cooldown mechanism prevents repeated triggers from the same pose during a short interval. The result is then displayed in the dashboard, logged in the history panel, and reflected in the statistics card. This gives the user immediate confirmation that the action was executed successfully.

3.4 Gesture Detection and Classification Logic

The gesture engine uses MediaPipe Hands to locate hand landmarks and produce a structured representation of the hand. These landmarks include the wrist, finger joints, and fingertips. From those points, the system can estimate whether fingers are open or closed, whether the palm is facing the camera, and whether the hand is moving in a swipe-like pattern.

To improve speed, the system does not analyze every frame at full cost. It may skip frames or use reduced-size images during the detection step. This kind of optimization is helpful because the dashboard should stay responsive even on modest hardware. The result of detection is cached where appropriate so that repeated calculations do not overload the pipeline.

The classification rules are intentionally simple. A small gesture vocabulary reduces ambiguity and makes the system easier to use. For example, an open palm may represent home, two raised fingers may represent next, and a single raised finger may represent previous. If the posture is not clear, a swipe fallback can be used. This

combination gives the system enough flexibility while keeping the mapping understandable.

A cooldown timer is also part of the logic. Without cooldown, a user who holds the same pose for a fraction of a second too long could trigger multiple commands. The cooldown mechanism ensures that one clear gesture results in one action, which makes the interaction more stable and predictable.

3.5 List of Modules

- Configuration Module
- Camera Capture and Streaming Module
- Gesture Detection Module
- Gesture Classification Module
- Slide Control Module
- Dashboard and Frontend Module
- History and Analytics Module
- Admin and Settings Module

3.5 Modules Description

Configuration Module

This module stores important values such as camera index, frame size, confidence thresholds, cooldown duration, swipe sensitivity, and the gesture-to-key mapping. Keeping these values in one place makes the project easier to tune and maintain. It also allows future users to adjust behavior without rewriting core logic.

Module	Description
Configuration	Stores camera, threshold, cooldown, and mapping values.
Camera Streaming	Captures video and sends live frames to the browser.
Gesture Engine	Detects hand landmarks and classifies gestures.
Slide Control	Turns gestures into keyboard actions.
Dashboard	Displays live feed, actions, and status.
History & Analytics	Keeps logs and usage statistics for review.
Admin Tools	Supports camera switching and settings management.

Camera Capture and Streaming Module

This module opens the webcam or external camera, captures frames continuously, and sends them to the browser through WebSocket communication. It also handles retry logic so the system can recover when a camera source is temporarily unavailable. This helps the application remain usable in changing environments.

Gesture Detection Module

This module uses MediaPipe Hands to identify hand landmarks from each frame. It applies performance optimization by processing only selected frames and by downscaling the input for faster inference. Cached results may be reused when necessary so that the dashboard remains smooth.

Gesture Classification Module

This module decides whether a detected hand posture represents next, previous, home, or a swipe-based fallback action. The classification logic can combine finger count and wrist movement so that the system remains useful even when one type of gesture is not clear enough. The rules are compact enough for an academic project but flexible enough for improvement.

Slide Control Module

This module translates gesture names into operating system key presses using PyAutoGUI. It adds cooldown-based debouncing and records action history so that the same gesture does not fire continuously. This is the part of the system that connects recognition to real computer control.

Dashboard and Frontend Module

This module renders the visual interface in the browser using HTML, CSS, and JavaScript. It shows the live feed, current gesture, FPS, connection status, action log, and quick guide panels in a polished layout. The dashboard is what makes the system understandable to a user during live use.

History and Analytics Module

This module maintains recent gesture records and summary statistics such as total gestures, most used gesture, and action breakdown. The user can review earlier actions and understand how frequently each gesture is being used. This supports transparency and basic performance analysis.

Admin and Settings Module

This module allows system-level control of camera source switching, operational settings, and page updates. It is especially useful when the gesture controller is demonstrated in different environments or with different hardware

sources. In a full deployment, it can also support role-based access and configuration control.

3.6 Gesture Mapping Rules

The mapping rules are intentionally small and readable. A clear mapping table reduces ambiguity and helps the user remember what each hand pose does. The system can later be expanded with more commands, but beginning with a short list keeps the first version dependable.

3.8 Data Flow and State Handling

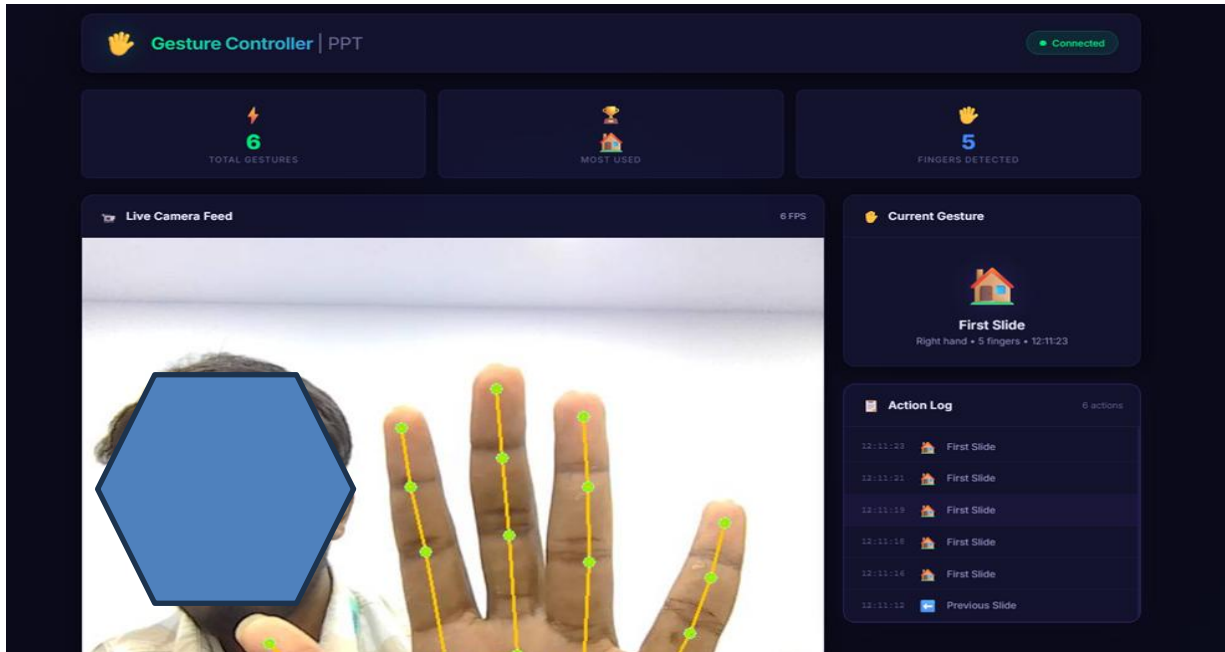
The system carries a small amount of state to support smooth interaction. It stores the current gesture, the last recognized action, the timestamp of the previous trigger, and a few counters for dashboard statistics. These values help the interface show a meaningful history without overwhelming the backend with unnecessary detail.

The data flow is simple: camera frame to detection, detection to classification, classification to action, and action to dashboard update. This simple sequence is important because it keeps the system understandable and reduces the chance of errors. For a real-time application, clarity in the flow is often more useful than a large number of complicated features.

CHAPTER 4

RESULTS AND DISCUSSION

4.1 Screenshots



This section describes the visible outputs of the developed hand mapping system. The interface is designed as a modern browser dashboard so that the user can observe the camera feed, detected gesture, and resulting action without switching windows. The goal is to make the system transparent and easy to inspect during live use.

The visual design uses a dark computer-vision theme with clear borders, live indicators, and summary cards. This aesthetic supports the technical nature of the project while keeping the interface readable and professional. It also provides a strong sense of structure, which is important when the user is making decisions based on real-time feedback.

The dashboard layout is intentionally compact but informative. The live feed panel shows what the camera sees, the status card shows the latest gesture, the log

panel records the action history, and the statistics card summarizes usage. Together these components help the user understand how the system behaves in actual operation.

4.2 Results Obtained

The developed system successfully demonstrates that hand mapping can be used as a dependable control method for slide navigation and other keyboard-driven tasks. Real-time gesture recognition works through webcam input, and the browser dashboard makes the system easy to inspect. The user can see whether the system detected a hand, which gesture was identified, and which action was fired.

WebSocket-based streaming improves responsiveness because the camera frames and gesture metadata are delivered continuously rather than through repeated page refreshes. The dashboard remains synchronized with the backend, and the interface can show updated statistics, action logs, and status indicators with low visible delay. This creates a much smoother interaction than a static script-based solution.

The performance is improved by practical optimization choices such as processing only selected frames, resizing frames to a smaller inference size, and reusing recent detection results when a full recalculation is not needed. These steps reduce overhead while preserving enough accuracy for presentation control. The system therefore works well as a lightweight demonstration of AI-edge style interaction.

From a usability point of view, the result is a clear and friendly interaction pattern. The gesture guide explains what the system expects, the action log confirms what happened, and the statistics panel shows overall usage. This combination

reduces uncertainty and makes the controller easier to learn during short practice sessions.

4.3 Discussion

The project shows that a hand gesture controller is more than a recognition model. It is a complete interaction system that includes feedback, logging, configuration, and recovery. If the backend only detected gestures without showing status, users would find it difficult to trust the result. The browser interface therefore plays a major role in making the overall system useful.

The modular backend design also supports future growth. A new gesture can be added by extending the mapping table and classification logic. The dashboard can later include more analytics, such as the number of gestures per session, average response time, or usage by camera source. This means the current implementation is not only functional but also expandable.

There are still a few limitations. Gesture accuracy can be affected by lighting, camera angle, hand occlusion, or fast motion. Swipe-based interaction may overlap with finger-count gestures if the classification rules are not carefully tuned. These issues do not prevent the system from working, but they show where future refinement would be useful.

The system also shows why web-based visualization matters in AI projects. Many users can understand a browser dashboard more easily than a terminal window or log file. When the camera feed, gesture status, and action history are displayed together, the system becomes easier to validate, demonstrate, and explain in an academic setting.

4.4 Advantages of the Proposed System

- Uses affordable and commonly available hardware.
- Provides real-time visual feedback through the browser.
- Reduces the need for keyboards, remotes, and touchpads during live sessions.
- Keeps the interface simple enough for first-time users.
- Supports modular extension with new gestures and actions.
- Offers a practical example of edge-style AI interaction.

4.5 Limitations of the Proposed System

- Recognition can be affected by poor lighting and background clutter.
- Very fast hand motion may reduce gesture stability.
- The current gesture set is intentionally small, so it must be extended for broader use.
- Camera hardware quality can influence frame clarity and detection confidence.
- The system needs proper tuning for different environments.

4.6 Performance Observation Table

Condition	Observed Behavior	Impact	Remark
Good lighting	Landmarks are detected consistently	High reliability	Recommended for best results
Moderate lighting	Detection remains usable with small errors	Acceptable reliability	Normal classroom use is possible
Low lighting	Landmark confidence may decrease	Some instability	Needs improvement or adjustment
Fast hand movement	Frames may miss a pose transition	Possible false trigger reduction	Cooldown helps manage this
Stable posture	Most accurate gesture classification	Strong reliability	Ideal for navigation commands

CHAPTER 5

CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

Enhancing Human-Computer Interaction with Hand Mapping (Google AI Edge) presents a practical and modern approach to gesture-based control. The project proves that a webcam, a gesture engine, and a browser dashboard can be combined to create an interaction method that is smooth, affordable, and easy to understand. By mapping simple hand poses to slide navigation keys, the system reduces manual effort and supports more natural presentation control.

The implementation successfully integrates MediaPipe Hands, OpenCV, FastAPI, WebSocket communication, and PyAutoGUI into one modular application. The frontend provides live visibility, action history, and system feedback, while the backend performs real-time gesture processing and key simulation. This balance between recognition and usability makes the project suitable for demonstrations, academic submission, and further extension.

The system also highlights the value of edge-style interaction design. Instead of depending on heavy cloud computation, the project performs detection close to the camera source and returns immediate results to the user. That makes the response time acceptable for real use and keeps the architecture straightforward. The project therefore demonstrates a useful real-world application of AI-assisted human-computer interaction.

5.2 Future Scope

Future versions of the system can include more gestures such as volume control, full-screen toggle, annotation mode, last-slide navigation, and browser tab switching. Expanding the gesture vocabulary would make the controller more versatile and allow it to support a wider range of presentation and accessibility tasks.

A mobile-friendly version, better analytics, secure user profiles, and HTTPS or WebSocket security can further improve deployment quality. The system could also be expanded with gesture calibration so that users can adjust thresholds according to their hand size, camera distance, and room lighting. These enhancements would make the platform more robust and suitable for broader use.

The project could also be connected to smart classroom or conference-room environments. In such settings, the same gesture control could be used to manage slides, media, or interactive teaching modules. With a larger command set and stronger analytics, the platform could become a flexible touchless control layer for many applications.

5.3 Final Remark

The final outcome of the project is not only a working software application but also a demonstration of how modern web and computer vision tools can solve a practical interaction problem. The system shows that a simple idea, when designed carefully, can improve the way people present, teach, and demonstrate digital content.

In conclusion, the proposed hand mapping system supports educational value, digital innovation, and accessible interaction. It shows how a compact real-time computer vision solution can contribute to a better user experience and a more inclusive use of technology in everyday settings.

REFERENCES

1. M. Oudah, A. Al-Naji, and J. Chahl, “Hand Gesture Recognition Based on Computer Vision: A Review of Techniques,” **Journal of Imaging**, vol. 6, no. 8, art. 73, 2020.
2. A.Jaramillo-Yáñez, M. E. Benalcázar, and E. Mena-Maldonado, “Real-Time Hand Gesture Recognition Using Surface Electromyography and Machine Learning: A Systematic Literature Review,” **Sensors**, vol. 20, no. 9, art. 2467, 2020.
3. A.Haria, A. Subramanian, N. Asokkumar, S. Poddar, and J. S. Nayak, “Hand Gesture Recognition for Human Computer Interaction,” **Procedia Computer Science**, vol. 115, pp. 367–374, 2017.
4. D.-S. Tran, N.-H. Ho, H.-J. Yang, E.-T. Baek, S.-H. Kim, and G. Lee, “Real-Time Hand Gesture Spotting and Recognition Using RGB-D Camera and 3D Convolutional Neural Network,” **Applied Sciences**, vol. 10, no. 2, art. 722, 2020.
5. J. P. Sahoo, A. J. Prakash, P. Pławiak, and S. Samantray, “Real-Time Hand Gesture Recognition Using Fine-Tuned Convolutional Neural Network,” **Sensors**, vol. 22, no. 3, art. 706, 2022.
6. I. Alabdullah, H. Ansar, N. A. Mudawi, A. Alazeb, A. Alshahrani, S. S. Alotaibi, and A. Jalal, “Smart Home Automation-Based Hand Gesture Recognition Using Feature Fusion and Recurrent Neural Network,” **Sensors**, vol. 23, no. 17, art. 7523, 2023.
7. A.Mujahid, M. J. Awan, A. Yasin, M. A. Mohammed, R. Damaševičius, R. Maskeliūnas, and K. H. Abdulkareem, “Real-Time Hand Gesture Recognition

- Based on Deep Learning YOLOv3 Model,” **Applied Sciences**, vol. 11, no. 9, art. 4164, 2021.
8. S. Ahmed, M. M. Hasan, A. K. M. B. R. Ahmed, and others, “Hand Gestures Recognition Using Radar Sensors for Human–Computer Interaction: A Review,” **Remote Sensing**, vol. 13, no. 3, art. 527, 2021.
 9. S. Zhao, M. A. et al., “Hand Gesture Recognition on a Resource-Limited Interactive Wristband,” **Sensors**, vol. 21, no. 17, art. 5713, 2021.
 10. S. K. Krishna and N. Sinha, “Gestop: Customizable Gesture Control of Computer Systems,” in **Proc. ACM CODS-COMAD**, pp. 405–409, 2021.
 11. F. Zhang, V. Bazarevsky, A. Vakunov, A. Tkachenka, G. Sung, C.-L. Chang, and M. Grundmann, “MediaPipe Hands: On-device Real-time Hand Tracking,” in **Proc. IEEE/CVF CVPR Workshops**, pp. 1–10, 2020.
 12. G. Sung, K. Sokal, E. Uboweja, V. Bazarevsky, J. Baccash, E. G. Bazavan, C.-L. Chang, and M. Grundmann, “On-device Real-time Hand Gesture Recognition,” **arXiv preprint* arXiv:2111.00038*, 2021.
 13. E. Uboweja, D. Tian, Q. Wang, Y.-C. Kuo, J. Zou, L. Wang, G. Sung, and M. Grundmann, “On-Device Real-Time Custom Hand Gesture Recognition,” in **Proc. IEEE/CVF ICCV Workshops**, pp. 4273–4277, 2023.
 14. A. Kapitanov, K. Kvanchiani, A. Nagaev, R. Kraynov, and A. Makhliarchuk, “HaGRID — HAnd Gesture Recognition Image Dataset,” in **Proc. IEEE/CVF WACV**, pp. 4572–4581, 2024.
 15. Z. Ren, J. Yuan, J. Meng, and Z. Zhang, “Robust Part-Based Hand Gesture Recognition Using Kinect Sensor,” **IEEE Transactions on Multimedia**, vol. 15, pp. 1110–1120, 2013.

CHAPTER 7 – MAPPING

This chapter presents the mapping of the Vision-Auth project outcomes to Programme Outcomes (PO) as defined by the National Board of Accreditation (NBA) and to the United Nations Sustainable Development Goals (SDG).

7.1 Programme Outcome (PO) Mapping

The Vision-Auth project addresses multiple Programme Outcomes of the B.Tech Artificial Intelligence and Data Science programme. The following table presents the PO mapping with justification for each applicable outcome.

S.No	PO	Program Outcome	Relevance to the Project
1	PO1	Engineering Knowledge: Apply knowledge of mathematics, science, computing, and engineering fundamentals to solve complex problems.	The project applies computer vision, web development, and AI-edge concepts to develop a gesture-based control system.
2	PO2	Problem Analysis: Identify, formulate, and analyze complex engineering problems using research knowledge.	The project analyzes the difficulty of traditional presentation control and proposes gesture-based interaction as a solution.

S.No	PO	Program Outcome	Relevance to the Project
3	PO3	Design/Development of Solutions: Design systems to meet user needs considering constraints.	A complete system with frontend, backend, and gesture recognition is designed for real-time interaction.
4	PO4	Conduct Investigations: Use experiments, data analysis, and interpretation.	The system is tested using real-time webcam input and gesture accuracy validation.
5	PO5	Engineering Tool Usage: Use modern tools and technologies effectively.	Tools like MediaPipe, OpenCV, FastAPI, and PyAutoGUI are used to build the system.
6	PO6	The Engineer and Society: Evaluate societal impact of engineering solutions.	The system improves accessibility and productivity in presentations and training environments.
7	PO7	Ethics: Apply ethical principles and professional responsibility.	The project ensures responsible use of webcam data and avoids storing sensitive information.

S.No	PO	Program Outcome	Relevance to the Project
8	PO8	Individual and Team Work: Work effectively in teams.	The project can be developed collaboratively with modular design and shared responsibilities.
9	PO9	Communication: Communicate effectively through reports and presentations.	Documentation, dashboard interface, and logs help communicate system behavior clearly.
10	PO10	Project Management and Finance: Apply management principles.	The project uses low-cost tools and follows structured development methodology.
11	PO11	Life-long Learning: Engage in continuous learning and adaptability.	The project involves learning modern AI, real-time systems, and web technologies.

7.2 Program Specific Outcomes (PSO)

S.No	PSO	Description	Relevance to the Project
1	PSO1	Apply programming and AI techniques to build intelligent systems.	The project uses MediaPipe and computer vision for gesture recognition.
2	PSO2	Analyze problems and develop efficient solutions using domain knowledge.	The project solves interaction challenges using gesture-based control logic.

7.3 Sustainable Development Goals (SDG) Mapping

S.No	SDG	Goal	Relevance to the Project
1	SDG 4	Quality Education	The system improves classroom and seminar presentations by enabling smooth, gesture-based interaction, making teaching more engaging and effective.
2	SDG 9	Industry, Innovation and Infrastructure	The project promotes innovation by applying AI and computer vision to develop a modern human-computer interaction system.
3	SDG 3	Good Health and Well-being	Supports touchless interaction, reducing physical strain and improving comfort during long usage sessions.
4	SDG 10	Reduced Inequalities	Helps users with limited mobility or difficulty using traditional input devices by providing an alternative interaction method.
5	SDG 8	Decent Work and Economic Growth	Enhances productivity in workplaces and presentations by enabling faster and more natural system control.

CHAPTER VIII

CONFERENCE DETAILS

The project paper is proposed for submission to the 10th International Conference on Engineering, Technology and Science (ICETS'26), organized by Muthayammal College of Engineering, Rasipuram, Namakkal District, Tamil Nadu. This is a virtual conference scheduled to be held on 28th March 2026, Saturday. The conference is organized by the departments of CSE, AI & DS, ECE, EEE, and Agricultural Engineering. It provides a platform for students, researchers, and professionals to present their innovative ideas and technical work in the fields of engineering, technology, and science.

The conference aims to encourage research-based learning and the sharing of new developments in emerging technologies. It supports presentations and publications from academic and research communities. The event also offers opportunities for conference proceedings publication, and selected papers may be considered for journal publication based on peer review and relevance to the journal scope. The conference encourages interdisciplinary research and practical applications that solve real-world problems.

The registration fee for UG and PG students, research scholars, and academic delegates is ₹500 per team. For journal publication, an additional charge may apply. The important dates mentioned in the conference notice include manuscript submission on 25.03.2026, review feedback on 26.03.2026, and manuscript registration on 27.03.2026. The organizers have also indicated that conference proceedings will carry an ISBN number, and some selected papers may be published in a journal depending on the quality, scope, and peer review comments.

This conference is suitable for our project because it accepts technical papers in areas related to artificial intelligence, computer vision, human-computer interaction, and real-time applications. Our proposed work on AI-based hand gesture recognition fits well within the scope of the conference, as it demonstrates an innovative touchless interaction system that can be applied to presentations, media control, smart devices, and other real-time environments. Participating in this conference will help us gain research exposure, improve our presentation skills, and add academic value .

MUTHAYAMMAL COLLEGE OF ENGINEERING

(Approved by AICTE, New Delhi & Affiliated to Anna University)

Rasipuram - 637 408, Namakkal Dist., Tamil Nadu.

CERTIFICATE OF PARTICIPATION

This is to certify that

Rajesh B

Kamaraj College of Engineering and Technology, Madurai

has presented a paper titled

Enhancing Human-Computer Interaction Using Real-Time Hand Gesture Recognition System

in

10th International Conference on Engineering Technology and Science (ICETS' 26)

organized by Muthayammal College of Engineering, Rasipuram,

Namakkal Dt., Tamil Nadu on 28th March 2026.



Paper ID
CS224

[Signature]
Chairperson

[Signature]
Dr. C. NAGARAJAN
Convener

[Signature]
Dr. P. VENUGOPAL
Principal



MUTHAYAMMAL COLLEGE OF ENGINEERING

(Approved by AICTE, New Delhi & Affiliated to Anna University)

Rasipuram - 637 408, Namakkal Dist., Tamil Nadu.

CERTIFICATE OF PARTICIPATION

This is to certify that

Harish Gowtham B

Kamaraj College of Engineering and Technology, Madurai

has presented a paper titled

Enhancing Human-Computer Interaction Using Real-Time Hand Gesture Recognition System

in

10th International Conference on Engineering Technology and Science (ICETS' 26)

organized by Muthayammal College of Engineering, Rasipuram,

Namakkal Dt., Tamil Nadu on 28th March 2026.



Paper ID
CS224

[Signature]
Chairperson

[Signature]
Dr. C. NAGARAJAN
Convener

[Signature]
Dr. P. VENUGOPAL
Principal



ICETS
2026

ICETS
2026

MUTHAYAMMAL COLLEGE OF ENGINEERING

(Approved by AICTE, New Delhi & Affiliated to Anna University)

Rasipuram - 637 408, Namakkal Dist., Tamil Nadu.

CERTIFICATE OF PARTICIPATION

This is to certify that

Venkatesh M

Kamaraj College of Engineering and Technology, Madurai

has presented a paper titled

Enhancing Human-Computer Interaction Using Real-Time Hand Gesture Recognition System

in

10th International Conference on Engineering Technology and Science (ICETS' 26)

organized by Muthayammal College of Engineering, Rasipuram,

Namakkal Dt., Tamil Nadu on 28th March 2026.



Paper ID
CS224

[Signature]
28/3/26
Chairperson

[Signature]
Dr. C. NAGARAJAN
Convener

[Signature]
Dr. P. VENUGOPAL
Principal



ICETS
2026